

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
КИЇВСЬКИЙ НАЦІОНАЛЬНИЙ
УНІВЕРСИТЕТ БУДІВНИЦТВА І
АРХІТЕКТУРИ

Кафедра Кібербезпеки та комп'ютерної інженерії

ЛАБОРАТОРНА РОБОТА № 4

з курсу “ WEB-програмування ”

на тему: “Реалізація інтерактивних компонентів веб-сторінки засобами JavaScript (без використання додаткових бібліотек): маніпуляція DOM (Document Object Model), обробка подій та динамічне оновлення вмісту ”

Виконав

: група

бікс-23-1

Полікапов Антон Юрійович

Перевірів

: Маленко М.

В.

Мета роботи:Набути практичних навичок програмування клієнтської логіки засобами мови JavaScript; освоїти API роботи з DOM: вибірку елементів, їх створення, модифікацію та видалення; навчитися реєструвати обробники подій та реагувати на дії користувача; сформувати розуміння принципів розмежування відповідальності між HTML, CSS та JavaScript.

Завдання:

12	Сайт кінотеатру	Розклад сеансів з фільтром за жанром	click, change	querySelectorAll + forEach + classList для фільтрації карток
----	-----------------	--------------------------------------	---------------	--

Додаток 1

1. Структура HTML (Файл `index.html`)

HTML-документ створює скелет сторінки та визначає зміст сайту кінотеатру.

Мета-дані та підключення:

- Використовується кодування UTF-8 та налаштовано viewport для коректного відображення на мобільних пристроях.
- Підключено зовнішні стилі `styles.css` та скрипт `script.js` з атрибутом `defer` перед закриваючим тегом `</body>`, що забезпечує виконання коду після повної обробки HTML-структури.

Семантична розмітка:

- **`<header>`:** Містить назву кінотеатру "CinemaHall".
- **`<nav>`:** Головна навігація з посиланнями на розклад сеансів та інформаційні розділи.
- **`<main>`:** Основний контейнер, що використовує CSS Grid для логічного розміщення розкладу та бічної панелі.
- **`<aside>`:** Бокова панель, де розміщено лічильник знайдених фільмів та актуальні пропозиції.
- **`<footer>`:** Підвал сайту з інформацією про авторські права.

Функціонал фільтрації:

- **`#filter-controls`:** Містить елементи керування для фільтрації. Згідно з варіантом, використано випадальний список `<select>` з атрибутом `data-filter` та швидкі кнопки `<button>` з атрибутами `data-genre` для вибору жанру (наприклад, sci-fi, comedy).
- **`#movie-list` (gallery):** Контейнер з картками фільмів. Кожна картка (`.movie-card`) містить атрибут `data-genre`, який JavaScript використовує для співставлення з обраним фільтром.

2. Стилзація CSS (Файл `styles.css`)

Оформлення створює атмосферу сучасного кінотеатру та забезпечує адаптивність інтерфейсу.

Конфігурація (`:root`):

- Визначено змінні для кольорів: основний темно-графітовий (`--color-primary`) та акцентний золотистий (`--color-accent`), що імітує стилістику преміальних кінозалів.
- Встановлено стандартні параметри для радіусів закруглення (`--radius`) та відступів (`--spacing`) для забезпечення візуальної цілісності.

Макет (CSS Grid):

- `.grid-container`: Реалізує гнучку структуру за допомогою `grid-template-areas`, чітко розподіляючи зони для заголовка, навігації, розкладу фільмів, бокової панелі та футера.
- `.gallery`: Побудована на основі сітки, де кількість колонок динамічно змінюється залежно від ширини екрана (від 1 до 3)

Стили фільтрів та карток:

- `#genre-select`: Оформлено випадаючий список для події `change` з акцентом на зручність вибору на мобільних пристроях.
- `.filter-btn`: Кнопки мають стилізацію з переходом кольорів та клас `.active` для візуального зворотного зв'язку при кліку.
- `.hidden`: Критично важливий клас із властивістю `display: none !important`, який використовується JavaScript-кодом для миттєвого приховування фільмів, що не відповідають обраному жанру.
- `.movie-card`: Кожна картка сеансу має ефект масштабування при наведенні (`scale`), анімацію появи (`fadeIn`) та фіксовану висоту для постерів фільмів.

Адаптивність:

За допомогою медіа-запитів (`@media`) макет трансформується: на смартфонах елементи розміщуються в одну колонку, а на ПК — у складну сітку з бічною панеллю.

3. Логіка JavaScript (Файл `script.js`)

Скрипт забезпечує інтерактивність вибору фільмів та миттєве оновлення розкладу без перезавантаження сторінки.

Ініціалізація:

- Використовується обробник події `DOMContentLoaded`, що гарантує запуск скрипта лише після повного завантаження HTML-дереву.
- **Вибір елементів:** За допомогою методу `querySelectorAll` скрипт отримує доступ до масиву карток фільмів (`.movie-card`), кнопок швидкої фільтрації та випадаючого списку жанрів.

Обробка подій (Event Listening):

- Подія `change`: Реалізовано слухач для селекту жанрів. При виборі нового пункту спрацьовує функція фільтрації.
- Подія `click`: Для кнопок швидкого вибору додано обробники, які не лише фільтрують контент, а й візуально підсвічують активну кнопку через клас `.active`.

Алгоритм фільтрації:

- Зчитування даних: Скрипт отримує обраний жанр через властивість `value` селекту або атрибут `data-genre` кнопки.
- Маніпуляція DOM: За допомогою циклу `forEach` скрипт перебирає всі картки фільмів.
- Логіка відображення: Використовується властивість `classList`. Якщо жанр картки (отриманий через `dataset.genre`) збігається з обраним (або обрано "Всі"), метод `remove('hidden')` робить картку видимою. В іншому випадку метод `add('hidden')` приховує її.
- Динамічне оновлення: Після кожного циклу фільтрації скрипт перераховує кількість видимих фільмів та оновлює текстовий вміст (`textContent`) лічильника в бічній панелі, що відповідає вимогам щодо маніпуляції контентом.

Код(index.html)

```
<!DOCTYPE html>
<html lang="uk">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>CinemaHall – Розклад сеансів (Варіант 12)</title>
  <link rel="stylesheet" href="styles.css">
</head>
<body>

  <header class="header-area">
    <h1>CinemaHall</h1>
  </header>

  <nav class="nav-area" aria-label="Головна навігація">
    <ul>
      <li><a href="#home">Головна</a></li>
      <li><a href="#schedule">Розклад</a></li>
    </ul>
  </nav>

  <main class="grid-container">
    <article class="content-area">
      <section id="schedule">
        <h2>Розклад сеансів</h2>

        <div id="filter-controls">
          <label for="genre-select">Оберіть жанр:</label>
          <select id="genre-select" data-filter="genre">
            <option value="all">Всі жанри</option>
            <option value="action">Екшн</option>
            <option value="comedy">Комедія</option>
            <option value="drama">Драма</option>
            <option value="sci-fi">Фантастика</option>
          </select>

          <div id="quick-filters" style="margin-top: 10px;">
            <button class="filter-btn" data-genre="all">Всі</button>
            <button class="filter-btn" data-genre="sci-fi">Тільки
Фантастика</button>
          </div>
        </div>

        <div class="gallery" id="movie-list">
          <div class="card movie-card" data-genre="sci-fi">
            
            <h3>Інтерстеллар</h3>
            <p>Жанр: Фантастика</p>
            <p><strong>18:00 | 21:30</strong></p>
          </div>

          <div class="card movie-card" data-genre="comedy">
            
            <h3>Похмілля у Верасі</h3>
            <p>Жанр: Комедія</p>
            <p><strong>14:00 | 20:00</strong></p>
          </div>

          <div class="card movie-card" data-genre="action">
            
            <h3>Шалений Макс</h3>
            <p>Жанр: Екшн</p>
            <p><strong>16:45 | 22:00</strong></p>
          </div>
        </div>
      </section>
    </article>
  </main>
</body>
</html>
```

```
</div>

<div class="card movie-card" data-genre="drama">
  
  <h3>Кит</h3>
  <p>Жанр: Драма</p>
  <p><strong>12:00 | 15:30</strong></p>
</div>

<div class="card movie-card" data-genre="sci-fi">
  
  <h3>Дюна: Частина друга</h3>
  <p>Жанр: Фантастика</p>
  <p><strong>19:00 | 22:30</strong></p>
</div>
</div>
</section>
</article>

<aside class="sidebar-area">
  <h3>Сьогодні в прокаті</h3>
  <p>Не пропустіть головні прем'єри сезону!</p>
  <div id="status-message">Знайдено фільмів: <span
id="count">5</span></div>
</aside>
</main>

<footer class="footer-area">
  <p>&copy; 2026 CinemaHall. Програмна реалізація: [Ваше Прізвище]</p>
</footer>

<script src="script.js" defer></script>
</body>
</html>
```


Код (styles.css)

```
@import
url('https://fonts.googleapis.com/css2?family=Roboto:wght@400;700&display=swap');

:root {
  --color-primary: #1a1a1a;
  --color-accent: #f39c12;
  --color-bg: #f4f4f4;
  --color-text: #2c3e50;
  --radius: 8px;
  --spacing: 1.5rem;
  --color-danger: #e74c3c;
}

* { box-sizing: border-box; }
body, h1, h2, h3, p, ul { margin: 0; padding: 0; }

body {
  font-family: 'Roboto', sans-serif;
  background-color: var(--color-bg);
  color: var(--color-text);
  line-height: 1.6;
}

img {
  max-width: 100%;
  height: auto;
  display: block;
}

h1 { font-size: clamp(1.5rem, 5vw, 2.5rem); text-align: center; }

.grid-container {
  display: grid;
  gap: var(--spacing);
  padding: var(--spacing);
  max-width: 1200px;
  margin: 0 auto;
  grid-template-areas:
    "header"
    "nav"
    "content"
    "sidebar"
    "footer";
}

.header-area {
  grid-area: header;
  background: var(--color-primary);
  color: var(--color-accent);
  padding: 2rem;
  border-radius: var(--radius);
  border-bottom: 4px solid var(--color-accent);
}

.nav-area {
  grid-area: nav;
  background: white;
  padding: 1rem;
  border-radius: var(--radius);
  box-shadow: 0 2px 5px rgba(0,0,0,0.1);
}

.content-area { grid-area: content; }
```



```

.sidebar-area {
  grid-area: sidebar;
  background: white;
  padding: var(--spacing);
  border-radius: var(--radius);
}

.footer-area {
  grid-area: footer;
  text-align: center;
  padding: 2rem;
  background: var(--color-primary);
  color: white;
  border-radius: var(--radius);
}

nav ul { display: flex; justify-content: center; list-style: none; gap: 1.5rem;
flex-wrap: wrap; }
nav a { text-decoration: none; color: var(--color-primary); font-weight: bold;
transition: 0.3s; }
nav a:hover { color: var(--color-accent); }

#filter-controls {
  background: #fff;
  padding: 1.5rem;
  border-radius: var(--radius);
  margin-bottom: var(--spacing);
  border-left: 5px solid var(--color-accent);
}

#genre-select {
  padding: 8px 12px;
  border-radius: 4px;
  border: 1px solid #ccc;
  font-size: 1rem;
  margin-left: 10px;
}

#quick-filters {
  display: flex;
  gap: 10px;
  margin-top: 15px;
}

.filter-btn {
  padding: 8px 16px;
  border: 2px solid var(--color-primary);
  background: transparent;
  color: var(--color-primary);
  cursor: pointer;
  border-radius: 4px;
  font-weight: bold;
  transition: 0.3s;
}
*/
.filter-btn.active, .filter-btn:hover {
  background: var(--color-primary);
  color: var(--color-accent);
}

.hidden {
  display: none !important;
}

.gallery {
  display: grid;

```

```

    grid-template-columns: 1fr;
    gap: var(--spacing);
}

.movie-card {
    background: white;
    padding: 1rem;
    border-radius: var(--radius);
    border: 1px solid #ddd;
    text-align: center;
    transition: transform 0.3s ease, box-shadow 0.3s ease;
    animation: fadeIn 0.5s ease-in;
}

.movie-card:hover {
    transform: scale(1.03);
    box-shadow: 0 10px 20px rgba(0,0,0,0.15);
}

.movie-card img {
    width: 100%;
    height: 300px;
    object-fit: cover;
    border-radius: calc(var(--radius) - 2px);
    margin-bottom: 1rem;
}

.movie-card h3 { color: var(--color-primary); margin-bottom: 0.5rem; }
.movie-card p { font-size: 0.9rem; color: #666; }
.movie-card strong { color: var(--color-danger); }

@keyframes fadeIn {
    from { opacity: 0; }
    to { opacity: 1; }
}

/* 6. АДАПТИВНІСТЬ */
@media (min-width: 768px) {
    .grid-container {
        grid-template-columns: 1fr 280px;
        grid-template-areas:
            "header header"
            "nav nav"
            "content sidebar"
            "footer footer";
    }
    .gallery { grid-template-columns: repeat(2, 1fr); }
}

@media (min-width: 1024px) {
    .gallery { grid-template-columns: repeat(3, 1fr); }
}

```

Код ([script.js](#))

```
document.addEventListener('DOMContentLoaded', () => {  
  // 1. Ініціалізація та вибірка елементів [cite: 40]  
  const genreSelect = document.getElementById('genre-select');  
  const quickFilterButtons = document.querySelectorAll('.filter-btn');  
  const movieCards = document.querySelectorAll('.movie-card');  
  const counterDisplay = document.getElementById('count');  
  
  /**  
   * Основна функція фільтрації  
   * @param {string} selectedGenre - Жанр для фільтрації  
   */  
  const filterMovies = (selectedGenre) => {  
    let visibleCount = 0;  
  
    movieCards.forEach(card => {  
      // Отримуємо жанр конкретної картки через dataset  
      const cardGenre = card.dataset.genre;  
  
      // Логіка відображення: якщо обрано "all" або жанри збігаються  
      if (selectedGenre === 'all' || cardGenre === selectedGenre) {  
        card.classList.remove('hidden'); // Показуємо [cite: 14, 69]  
        visibleCount++;  
      } else {  
        card.classList.add('hidden'); // Приховуємо [cite: 14, 69]  
      }  
    });  
  
    // Динамічне оновлення вмісту (лічильника)  
    if (counterDisplay) {  
      counterDisplay.textContent = visibleCount;  
    }  
  
    console.log(`Фільтрація завершена. Відображено фільмів: ${visibleCount}`);  
  };  
  
  // 2. Реєстрація обробника події 'change' для <select> [cite: 19, 72]  
  genreSelect.addEventListener('change', (event) => {  
    const genre = event.target.value;  
    filterMovies(genre);  
  });  
});
```

```
// Скидаємо активний стан кнопок при зміні селекту
quickFilterButtons.forEach(btn => btn.classList.remove('active'));
});

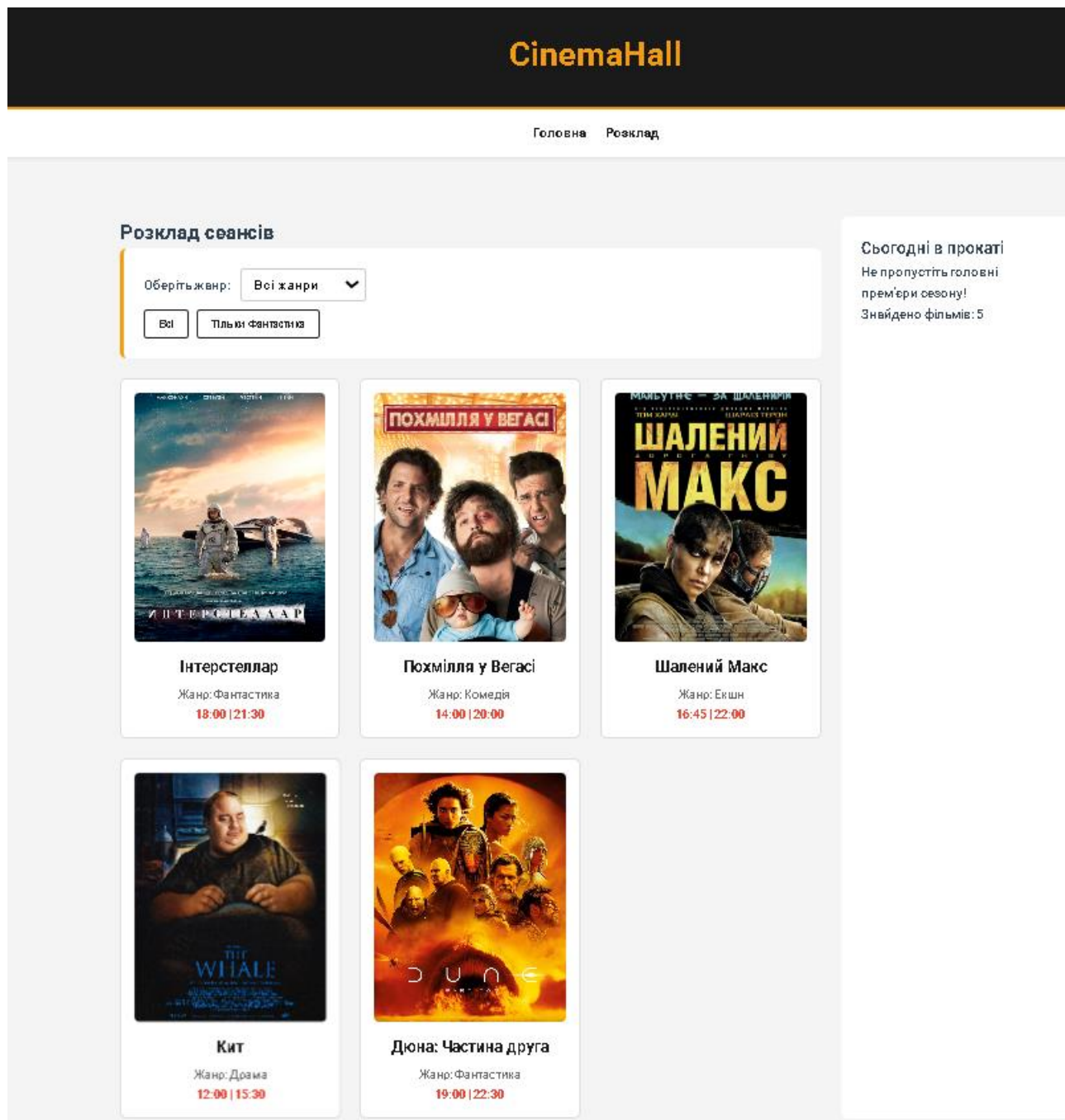
// 3. Реєстрація обробників події 'click' для кнопок [cite: 19, 72]
quickFilterButtons.forEach(button => {
  button.addEventListener('click', () => {
    // Візуальне перемикання активного класу [cite: 14, 69]
    quickFilterButtons.forEach(btn => btn.classList.remove('active'));
    button.classList.add('active');

    const genre = button.getAttribute('data-genre');

    // Синхронізуємо випадаючий список
    genreSelect.value = genre;

    filterMovies(genre);
  });
});

// Початковий підрахунок при завантаженні
filterMovies('all');
});
```



Відповіді на питання

1. Що таке DOM і як він пов'язаний зі структурою HTML-документа? Різниця між методами вибору елементів.

- **DOM (Document Object Model)** — це об'єктне представлення HTML-документа у вигляді дерева, де кожен тег є окремим об'єктом (вузлом), яким можна керувати через JavaScript.
- **Зв'язок:** Браузер перетворює статичний текст у `index.html` на динамічне дерево DOM, що дозволяє скриптам змінювати вміст, структуру та стилі сторінки в реальному часі.
- **document.querySelector():**
 - Повертає **перший** знайдений елемент, що відповідає вказаному CSS-селектору.
 - Якщо нічого не знайдено, повертає `null`.
- **document.querySelectorAll():**
 - Повертає **статичний список усіх** елементів (NodeList), що відповідають

селектору.

- Саме цей метод використано у `script.js` для отримання всіх кнопок `.filter-btn` та карток `.card`.

2. Різниця між `textContent` та `innerHTML`. Коли уникати `innerHTML`?

- **`textContent`:**
 - Отримує або встановлює **чистий текст** всередині вузла, ігноруючи будь-які HTML-теги.
 - Це безпечніший та швидший метод для роботи з текстом.
- **`innerHTML`:**
 - Отримує або встановлює **повний HTML-код** всередині елемента.
 - Дозволяє динамічно створювати нові теги всередині існуючих.
- **Коли уникати:** Слід уникати `innerHTML`, коли ви вставляєте дані, отримані від користувача. Це створює ризик **XSS-атак** (міжсайтового скриптингу), оскільки злоумисник може вставити шкідливий скрипт `<script>...` безпосередньо у вашу сторінку.

3. Що таке event-делегування?

- **Визначення:** Це техніка, при якій замість того, щоб вішати обробник подій на кожен окремий дочірній елемент, ми вішаємо **один обробник на їхнього спільного батька**. Подія "спливає" (bubbling) від дитини до батька, де ми її перехоплюємо.
- **Коли необхідно:** Коли на сторінці багато однотипних елементів або коли елементи додаються динамічно після завантаження сторінки.
- **Приклад:** У вашому `script.js` замість циклу `forEach` для кожної кнопки, можна було б повісити один клік на `#filter-container` і через `event.target` перевіряти, на яку саме кнопку натиснули.

4. Для чого використовується `event.preventDefault()`?

- **Призначення:** Метод скасовує **стандартну поведінку браузера** для певної події, не зупиняючи подальше розповсюдження самої події.
- **Два приклади використання:**
 1. **Форми:** Скасування перезавантаження сторінки при натисканні кнопки "Надіслати", щоб обробити дані через JavaScript.
 2. **Посилання:** Скасування переходу за адресою `href` при кліку на посилання (наприклад, щоб відкрити модальне вікно замість переходу на нову сторінку).

5. Різниця між методами `classList`.

Ці методи використовуються для маніпуляції CSS-класами, як це зроблено у вашому проекті зі стилями `active` та `hidden`:

- **`classList.add()`:** Додає один або кілька класів. Доцільно, коли потрібно гарантовано приховати елемент (наприклад, додавання `.hidden`).
- **`classList.remove()`:** Видаляє класи. Використовується для активації елементів (наприклад, видалення `.hidden` у `script.js`).
- **`classList.toggle()`:** Додає клас, якщо його немає, і видаляє, якщо він є. Ідеально підходить для перемикачів (наприклад, кнопка "Темна тема" або "Показати/Сховати деталі").

1. Що таке `localStorage`, його обмеження та типи даних?

- **localStorage:** Це об'єкт веб-сховища, який дозволяє зберігати дані в браузері користувача **без терміну дії**. Дані зберігаються навіть після закриття вкладки або перезапуску браузера.
- **Обмеження:**
 - **Об'єм:** Зазвичай обмежений ~5-10 МБ на домен.
 - **Синхронність:** Операції читання/запису блокують основний потік виконання (не підходить для величезних об'ємів даних).
 - **Безпека:** Доступний для будь-якого скрипта на сторінці (не можна зберігати паролі або токени).
- **Типи даних:** Підтримує лише **рядки (strings)**. Щоб зберегти об'єкт або масив, їх потрібно перетворити на рядок через **JSON.stringify()**, а при читанні — назад через **JSON.parse()**.